Node.js — Zero to Master

22 topics, grouped as Beginner → Intermediate → Advanced → Master. Work through the topics in order; type every code example yourself and finish the practice tasks.

Contents

Beginner (6 topics)

01 Introduction and Installation

02 npm and package.json

03 Modules: CommonJS and ES Modules

04 Core Modules

05 File System

06 Environment Variables and Process

Intermediate (6 topics)

07 Events and EventEmitter

08 Asynchronous Patterns

09 HTTP Server Without Frameworks

10 Streams and Buffers

11 Working with Databases

12 Error Handling and Logging

Advanced (6 topics)

13 The Event Loop and Performance

14 Worker Threads and Cluster

15 Authentication: JWT and Sessions

16 Security Best Practices

17 Testing

18 WebSockets and Real-time

Master (4 topics)

19	Architecture and Project Structure
20	Docker, Deployment and CI/CD
21	Monitoring and Scaling
22	Real Project: REST API with Auth and DB

Beginner Level

01. Introduction and Installation

Node.js runs JavaScript outside the browser using the V8 engine, with a non-blocking I/O model. Install the LTS version and run files with `node`.

```
node -v
node app.js
```

02. npm and package.json

npm installs packages. `package.json` lists dependencies and scripts; `package-lock.json` pins versions.

```
npm init -y
npm i express
npm i -D nodemon
```

03. Modules: CommonJS and ES Modules

Use `import/export` (set `"type": "module"`) or `require/module.exports`.

```
import { readFile } from 'node:fs/promises';
```

04. Core Modules

`fs`, `path`, `os`, `url`, `crypto`, `http`, `events`, `util`, `stream`.

```
import path from 'node:path';
console.log(path.join('a', 'b', 'c.txt'));
```

05. File System

Read and write files with `fs/promises`; avoid blocking sync methods in servers.

```
import fs from 'node:fs/promises';
await fs.writeFile('a.txt', 'Hello');
console.log(await fs.readFile('a.txt', 'utf8'));
```

06. Environment Variables and Process

Read config from `process.env`; use `.env` files with `--env-file` or dotenv; handle `process.argv`.

```
node --env-file=.env app.js
console.log(process.env.PORT ?? 3000);
```

Intermediate Level

07. Events and EventEmitter

Many Node objects emit events; create your own with `EventEmitter`.

```
import { EventEmitter } from 'node:events';
const e = new EventEmitter();
e.on('order', id => console.log('order', id));
e.emit('order', 7);
```

08. Asynchronous Patterns

Callbacks, promises and `async/await`; `util.promisify`; never block the event loop.

```
const data = await fs.readFile('x.json', 'utf8');
```

09. HTTP Server Without Frameworks

`http.createServer` handles requests and responses; frameworks like Express build on it.

```
import http from 'node:http';
http.createServer((req, res) => res.end('Hello')).listen(3000);
```

10. Streams and Buffers

Streams process data in chunks (files, HTTP); `pipeline` connects them safely.

```
import { pipeline } from 'node:stream/promises';
import { createReadStream, createWriteStream } from 'node:fs';
await pipeline(createReadStream('in'), createWriteStream('out'));
```

11. Working with Databases

Use drivers/ORMs: `mysql2`, `pg`, `mongodb`/Mongoose, Prisma. Always use parameterised queries.

```
const [rows] = await pool.query('SELECT * FROM users WHERE id = ?', [id]);
```

12. Error Handling and Logging

Catch errors in async code, handle `unhandledRejection`, use structured logging (pino, winston).

```
process.on('unhandledRejection', err => { console.error(err); process.exit(1); });
```

Advanced Level

13. The Event Loop and Performance

Phases: timers, poll, check, close; `process.nextTick` and microtasks run first. Avoid CPU work on the main thread.

```
setImmediate(() => console.log('check'));
process.nextTick(() => console.log('tick'));
```

14. Worker Threads and Cluster

`worker_threads` run CPU-heavy tasks in parallel; `cluster` or PM2 use all CPU cores.

```
import { Worker } from 'node:worker_threads';
new Worker('./heavy.js');
```

15. Authentication: JWT and Sessions

Hash passwords with bcrypt/argon2, issue JWTs or session cookies, and verify on each request.

```
const token = jwt.sign({ sub: user.id }, process.env.JWT_SECRET, { expiresIn: '1h' });
```

16. Security Best Practices

Validate input (Zod), rate limit, use Helmet headers, CORS configuration, keep dependencies patched, never commit secrets.

```
npm audit
npm i helmet cors express-rate-limit
```

17. Testing

Built-in `node:test` or Vitest/Jest; supertest for APIs; mock external services.

```
import test from 'node:test';
import assert from 'node:assert';
test('adds', () => assert.equal(1 + 1, 2));
```

18. WebSockets and Real-time

Use `ws` or Socket.IO for chat, notifications and live dashboards.

```
io.on('connection', socket => socket.on('msg', m => io.emit('msg', m)));
```

Master Level

19. Architecture and Project Structure

Layers: routes, controllers, services, repositories; dependency injection, configuration, and clean separation of concerns.

```
src/
  routes/ controllers/ services/ repositories/ config/
```

20. Docker, Deployment and CI/CD

Containerise Node apps with multi-stage Dockerfiles; run behind Nginx; use PM2/systemd; deploy via CI/CD.

```
FROM node:22-alpine
WORKDIR /app
COPY package*.json ./
RUN npm ci --omit=dev
COPY . .
CMD ["node", "src/server.js"]
```

21. Monitoring and Scaling

Health checks, metrics (Prometheus), tracing (OpenTelemetry), caching with Redis, queues (BullMQ), horizontal scaling.

```
const queue = new Queue('emails'); await queue.add('welcome', { to });
```

22. Real Project: REST API with Auth and DB

CRUD API with validation, JWT auth, PostgreSQL/MongoDB, tests, Docker and deployment.

Interview Question Bank

Q1. What is Node.js?
A JavaScript runtime built on V8 with an event-driven, non-blocking I/O model.

Q2. Explain the event loop.
It processes callbacks in phases so a single thread can handle many concurrent I/O operations.

Q3. CommonJS vs ES Modules?
CommonJS uses `require` (synchronous); ESM uses `import` (static, async-capable).

Q4. What are streams?
Interfaces for processing data piece by piece, reducing memory usage.

Q5. `process.nextTick` vs `setImmediate`?
`nextTick` runs before the event loop continues; `setImmediate` runs in the check phase.

Q6. How do you scale Node.js?
Cluster/PM2 on multiple cores, load balancers, worker threads, caching and horizontal scaling.

Q7. How do you handle errors in async code?
`try/catch` with `await`, `.catch()` on promises and a global handler as a safety net.

Q8. What is middleware?
A function in the request pipeline with access to request, response and `next`.

Q9. How do you secure a Node app?
Validation, hashing, HTTPS, Helmet, CORS, rate limiting, dependency audits.

Q10. What is the difference between `dependencies` and `devDependencies`?
Runtime packages vs tools used only in development/build.

Capstone Project Ideas

1. CLI notes app
2. REST API for a library
3. Real-time chat
4. File upload service
5. URL shortener with Redis